WhatsApp Photo Archive — Quickstart

A step-by-step guide that takes you from "I have an iPhone with WhatsApp" to "I can search every photo in my chat history from my laptop and my phone, anywhere in the world".

Time:

- ~30–60 minutes of attention the first time.
- iMazing extraction runs ~5–30 minutes in the background.
- If you choose the iCloud Drive variant, the upload of a multi-GB photo library happens silently and can take hours to days. **You do not have to wait for that upload before continuing** — Apple keeps the local copy on disk while it uploads.
- Re-builds after a fresh iPhone backup later take ~5 minutes of active work.

You'll end with:

- A password-protected searchable archive of every photo in your WhatsApp history, on your own laptop.
- A small web viewer with thumbnails, captions search, chat and sender filters, and a lightbox.
- An encrypted Tailscale connection so the viewer opens on your iPhone from any network.
- (Optional) Shared access for one other trusted person — your partner — on their own laptop and phone.

See it before you build it (optional)

If you want to preview the viewer UI before doing any iPhone extraction, install the tool (Ste p 3 below) and run:

```
wa-photos demo
```

This builds a fake 12-photo archive in `~/.cache/whatsapp_photos/demo/` and opens the sam e viewer you'll use for your real data at `http://127.0.0.1:8765/`. Press Ctrl-C in the termin al to stop it. When you're ready, switch to the real workflow below.

What you'll need

- A Mac or Windows PC (the "laptop") with at least as much free disk space as the WhatsAp p library on your phone takes (often 50–250 GB). See Step 2 if your internal drive is tight.
- Your iPhone, with WhatsApp installed and chat history present.
- A USB / USB-C / Lightning cable to connect them.
- **iMazing** — the free trial lets you browse the backup; the Export App Data feature used in Step 2 requires a paid license. Equivalents: iExplorer, iBackup Viewer Pro.
- **Python 3.10 or newer** on the laptop. On a Mac, install it from `python.org/downloads` rather than relying on the version that ships with macOS — Apple's Python refuses package installs in recent versions, while the python.org installer "just works".
- **Tailscale** account (free for personal use), only if you want to reach the archive from your phone when away from home WiFi, or share access with another person.

Step 1 — Make a fresh iPhone backup

1. Connect the iPhone to the laptop with the cable. Unlock the phone.
2. The phone asks **"Trust This Computer?"** the first time — tap Trust and enter your iPhone passcode.
3. Open iMazing. Your phone appears in the left sidebar.
4. Click the phone, then Back Up Now (top toolbar).
5. Wait for the backup to finish (~5–60 minutes the first time, depending on size). If your iTunes / Finder backup is encrypted, iMazing asks for the backup password — enter it.

Tip: leave Automatic Backup on so future re-builds only need a fresh extraction (Step 2), no t a fresh backup.

Step 2 — Extract WhatsApp data with iMazing

⚠ **Don't confuse this with the WhatsApp viewer's Export button.** If you open Data → WhatsApp in iMazing you'll see a chat browser with an Export dropdown offering PDF / Excel / CSV / Text / RSMF / Attachments. **None of those are what we need** — they produce formatted reports, not the raw backup files. Close that view and follow the steps below.

1. With your iPhone selected, go to the **Overview** screen.
2. In the Quick Actions panel on the right, click **Manage Apps** (in older versions this lives i n the left sidebar as **Apps**).
3. Find **WhatsApp Messenger** in the app list.
4. **Right-click** it → **Extract App** (older versions label this Extract App Data — same feature). This menu item is the one we need — it's distinct from the Export button in the WhatsApp viewer. The right-click menu also contains Copy to Mac, Restore App, Uninstall App, etc.; ignore those.
5. When asked, make sure Include Media / Include Documents are **checked** — otherwise you get the database without the photos.
6. Choose a destination folder. The next subsection covers the two sensible options.
7. Wait for export. This is the slowest step — five to thirty minutes depending on chat-history size and how many photos you have.

> **Note:** Extract App Data requires a **paid iMazing license**. The > free trial only enables the viewer-style exports from Data → > WhatsApp, which is why those are the only ones visible if you haven't > bought a license yet.

When it finishes, your destination folder will contain something like:

```
wa-extract/
├── ChatStorage.sqlite
├── Library/
└── Message/
    └── Media/
        ├── 111@s.whatsapp.net/
        ├── 222@s.whatsapp.net/
        └── ...
```

Both `ChatStorage.sqlite` and the `Message/` tree must be present.

Option A — store on the laptop's internal drive

Pick this if you have enough free space (often 50–250 GB). Destination:

```
~/wa-extract/
```

On a Mac you can type ~ into iMazing's destination dialog; on Windows type `%USERPROFILE%\wa-extract\`. Remember the exact path — you'll re-enter it in Step 4.

Option B — store in iCloud Drive

Pick this if your internal drive is tight, or if you want a partner to access the same source files from their laptop (see Step 8).

Destination on Mac:

```
~/Library/Mobile Documents/com~apple~CloudDocs/wa-extract/
```

(In Finder this appears as **iCloud Drive → wa-extract**.) On Windows the equivalent is typically `%USERPROFILE%\iCloudDrive\wa-extract\`.

After extraction, iCloud starts uploading in the background. Apple's Optimize Mac Storage feature keeps frequently-used photos on disk and evicts rarely-used ones to the cloud after a while. The viewer in Step 6 will trigger automatic download of any photo it asks for that isn't currently on disk.

Strongly recommended if you choose Option B: turn on **Advanced Data Protection** on your iPhone (Settings → your name → iCloud → Advanced Data Protection). This makes iCloud Drive end-to-end encrypted — Apple itself can no longer read your photos. If you pla n to share with a partner (Step 8 / Option B-shared), they should enable ADP on their device s too, otherwise the shared folder downgrades to standard iCloud encryption.

⚠ Keep `photos.db` out of iCloud — see Step 4.

Option C — store on an external hard drive

Pick this if your internal drive is tight and you don't want to wait on an iCloud upload, or as a staging step before moving to iCloud later (see Migrating C → B below).

Destination on Mac:

```
/Volumes/<HDD-name>/wa-extract/
```

Replace `<HDD-name>` with whatever Finder shows for your drive in the sidebar. On Windows it's the drive letter, e.g. `E:\wa-extract\`.

Requirements:

- The drive must be formatted as **APFS** or **Mac OS Extended (HFS+)** on Mac, or **NTFS** o n Windows. **FAT32 won't work** (4 GB per-file limit). exFAT works in a pinch but APFS is safer with the many small `@s.whatsapp.net` folders.
- The drive must be **plugged in** during `wa-photos ingest` (Step 4) and during every `wa-photos serve` session (Step 6). Unplug it and the viewer can't load photos until you plug it back in.
- Keep `photos.db` itself on your **internal** disk (`~/photos.db`), not on the HDD — otherwise the server crashes the moment the drive ejects.

Migrating C → B (HDD → iCloud) later

Useful if you wanted to test everything locally first, then move the library to iCloud for partner-sharing or cross-device access:

1. In Finder, drag `wa-extract/` from the HDD into `iCloud Drive` in the sidebar. Wait until th e upload finishes (Finder shows a cloud icon next to each file — fully uploaded files lose the icon).
2. Re-run the `wa-photos ingest` command from Step 4, but with the **iCloud paths** (Option B). The old `photos.db` is overwritten with paths pointing at the iCloud copy. Takes ~1 minute — only re-indexing, not re-uploading.
3. Once you've verified the viewer works against iCloud, you can delete `wa-extract/` from the HDD.

Step 3 — Install the photo archive tool

Open a terminal:

- **Mac:** open Terminal (Cmd-Space → type "terminal" → Enter).
- **Windows:** open PowerShell (Win → type "powershell" → Enter).

The first time you run `git` on a fresh Mac, macOS pops up a dialog asking to install Command Line Tools. Click **Install** and wait ~5 minutes. After that, run:

```
git clone https://github.com/jwk-md-rad/whatsappmerger.git
cd whatsappmerger
python3 -m pip install -e .
```

Verify the install:

```
wa-photos --version
```

If you get the version number back, you're ready. If you get `command not found`, close and reopen the terminal and try again — your shell needs to re-read where commands live.

Step 4 — Build the searchable archive

Substitute the destination folder you chose in Step 2 for <wa-extract> below. Pick the form that matches the option you used.

If you chose Option A (extract on the laptop):

```
wa-photos ingest \
  ~/wa-extract/ChatStorage.sqlite \
  ~/wa-extract \
  -o ~/photos.db \
  --password
```

If you chose Option B (extract in iCloud Drive):

```
wa-photos ingest \
  "$HOME/Library/Mobile Documents/com~apple~CloudDocs/wa-extract/ChatStorage.sqlite" \
  "$HOME/Library/Mobile Documents/com~apple~CloudDocs/wa-extract" \
```

```
    -o ~/photos.db \
    --password
```

If you chose Option C (extract on an external HDD):

```
wa-photos ingest \
  /Volumes/<HDD-name>/wa-extract/ChatStorage.sqlite \
  /Volumes/<HDD-name>/wa-extract \
  -o ~/photos.db \
  --password
```

Make sure the HDD is plugged in before running this — and stays plugged in whenever you later run `wa-photos serve`. The `photos.db` output goes to your **internal** home directory, never on the HDD itself.

⚠ **Keep `photos.db` local — do not place it inside iCloud Drive.** SQLite databases an d cloud sync are a bad combination: if iCloud syncs a file mid-write, the database can corrupt. The photos live in iCloud; the small (~MB) database stays in your home directory.

When prompted, enter and confirm a **strong** password (4+ random words, or 12+ characters of mixed case, digits, and symbols). This is the password your phone's browser will ask for later. Forgot passwords cannot be recovered — write yours down somewhere safe.

The tool prints a summary like:

```
Ingested -> /Users/you/photos.db
  Photos inserted:        4821
  Chats:                  78
  Skipped (file missing): 12
  Skipped (not image):    3
  Elapsed:                47.21s
```

A few skipped rows are normal — those are messages whose attachment file isn't on disk anymore (e.g. expired view-once media). A skipped count in the hundreds or thousands usually means the extract in Step 2 didn't include media; re-run with Include Media / Includ e Documents checked in iMazing.

Step 5 — Install Tailscale on laptop and iPhone

1. **Laptop:** download from `tailscale.com/download` → install → sign in. Use whichever account is easiest (Google / Apple / Microsoft / GitHub) — you only need to remember it for the admin console later.
2. **iPhone:** App Store → search Tailscale → install → open → sign in with the **same** account.
3. The first time you start the iOS app it asks permission to install a VPN profile. Tap Allow. (Tailscale isn't a real VPN, but iOS classifies it that way.) Once the toggle at the top of th e Tailscale app is green, you're connected.

Verify both devices are on the same tailnet by opening `https://login.tailscale.com/admin/machines` — both should appear in the list.

No port-forwarding, no router configuration, no certificates.

Step 6 — Start the server

In the same terminal where you ran Step 4:

```
wa-photos serve ~/photos.db --host 0.0.0.0
```

You'll see a banner like this:

```
Serving photo archive on http://0.0.0.0:8765/
  LAN: http://192.168.1.42:8765/
  Tailscale IPv4: http://100.84.17.3:8765/
  Tailscale MagicDNS: http://mylaptop.tail-abcd.ts.net:8765/
(Password protection: ON — your browser will prompt for credentials.)
```

Verify on the laptop first (don't skip — saves debugging on the phone): open `http://127.0.0.1:8765/` in your laptop's browser. Enter the password. You should see the search page with your photos. If that works, the server is healthy.

Then copy the **Tailscale MagicDNS** URL — that's the one to use on the phone in Step 7 an d from anywhere outside the LAN.

Leave this terminal window open while you want the archive reachable. Closing it (or pressing Ctrl-C) stops the server.

Step 7 — Open the archive on your phone

1. On the iPhone, open the Tailscale app and make sure the toggle at the top is **on** (green).
2. Open Safari (or Chrome) and paste the MagicDNS URL from Step 6.
3. iOS prompts for credentials. Username can be anything (the tool ignores it). Enter the password you set in Step 4.
4. You're in. Use the search box, the chat / sender filters, the date range, and tap any thumbnail to view full-size.

Bookmark the URL so you don't have to re-type it. Add it to your home screen if you want a one-tap launcher.

Step 8 — Share the archive with a partner

You can let one other trusted person (your partner, a sibling) reach the same archive from their own laptop and phone, without exposing anything to the wider internet. There are two patterns; pick one.

Pattern A — they use your server (simpler)

Your laptop hosts; theirs is just a viewer. Works whenever your laptop is on.

On your side, once:

1. Open `https://login.tailscale.com/admin/users` while logged into your Tailscale account.
2. Click **Invite Users** and enter the other person's email address.
3. They receive an invitation email.

On their side, once:

1. They open the invitation, create or sign into a Tailscale account, and install Tailscale on their laptop (and optionally their phone) from `tailscale.com/download`.
2. Once Tailscale is running, their device is in your shared tailnet.

Using it:

1. They open the same MagicDNS URL from Step 6 in their browser, e.g. `http://yourlaptop.tail-abcd.ts.net:8765/`.
2. Their browser prompts for credentials. They use the same password you set in Step 4.

Constraint: your laptop must be on and `wa-photos serve` must be running. If your laptop i s off, they can't reach the archive.

Pattern B — either of you can host (resilient)

Both laptops can run the server independently, using a shared iCloud Drive folder as the common source of truth. Whoever's laptop is on at the moment is the active host.

This pattern requires **Option B (iCloud Drive)** from Step 2.

On your side, once:

1. In Finder, right-click the `wa-extract` folder inside iCloud Drive → Share → Share Folder... → add your partner's Apple ID with **Read only** access (safer; they can't accidentally edit th e source).
2. Complete the Tailscale invite from Pattern A above (steps 1–5) if you haven't already.

On their side, once:

1. They accept the iCloud Drive share. The `wa-extract` folder appears in their iCloud Drive on their Mac (Finder → iCloud Drive → Shared).
2. They turn on Advanced Data Protection on their iPhone, so the shared folder remains end-to-end-encrypted.
3. They install Python and the tool exactly as in Step 3 of this guide.
4. They run their own ingest, producing their own local `photos.db`:

```
      wa-photos ingest \
        "$HOME/Library/Mobile
Documents/com~apple~CloudDocs/wa-extract/ChatStorage.sqlite" \
        "$HOME/Library/Mobile Documents/com~apple~CloudDocs/wa-extract" \
        -o ~/photos.db \
        --password
```

They can choose the same password as yours (simpler) or a different one (each archive is independently gated).

Using it:

1. Whichever laptop is on, that person runs `wa-photos serve ~/photos.db --host 0.0.0.0` (Step 6) on their own machine.
2. From any device in the shared tailnet, both Tailscale MagicDNS URLs work — pick whichever points to a laptop that's currently on: `http://yourlaptop.tail-…/` or `http://herlaptop.tail-…/`.

When you make a fresh iPhone backup later (Step 1 again) and re-extract to the same iClou d folder, the updated files sync to both Macs. You each re-run `wa-photos ingest` locally to

refresh your own `photos.db`.

Tips & troubleshooting

- **`command not found: wa-photos`** after `pip install`. Close and reopen the terminal. The shell caches where commands live and needs a fresh start.
- **`pip install` fails with "externally-managed-environment"** on macOS. You're using Apple's bundled Python. Install Python from `python.org/downloads`, then run the install command using the new `python3`.
- **MagicDNS URL doesn't resolve** in the phone's browser. In the Tailscale admin console (login.tailscale.com/admin), go to DNS and confirm MagicDNS is enabled (it's on by defaul t for new tailnets). As a fallback, use the Tailscale IPv4 URL from the Step 6 banner.
- **A photo shows as broken** in the viewer. iMazing didn't pull every media file. Re-run the export from Step 2 with Include Media / Include Documents options checked, then re-run `wa-photos ingest`.
- **Forgot the password.** No recovery — the password is hashed. Reset with `wa-photos password set ~/photos.db`.
- **Want to refresh with new messages.** Repeat Steps 1 and 2 (fresh backup, fresh extract), then re-run `wa-photos ingest …` with the same output path. The old `photos.db` i s overwritten automatically.
- **Stop the server.** Press Ctrl-C in the terminal where it's running. Tailscale stays up; restart with the same command later.
- **iCloud sync is slow on first access.** With Optimize Mac Storage on, a photo that hasn't been opened in a long time may need to download from iCloud before it appears. This is normal — once downloaded, subsequent opens are instant.
- **Server keeps running but the URL stopped working.** Check that your laptop hasn't gone to sleep. Open System Settings → Battery (Mac) and disable sleep while plugged in, or just move the mouse to wake it. Tailscale reconnects automatically.
- **Browser warning "this is not private"** when visiting the URL. Expected — the server uses plain HTTP, but Tailscale's WireGuard layer below already encrypts every byte. Continue past the warning.

What stays private

- **The original photos** live wherever you put them — your laptop's disk, or your iCloud Drive folder. This tool uploads nothing on its own. If you chose Option B (iCloud), Apple's sync handles those files; turn on **Advanced Data Protection** to make even iCloud Drive end-to-end encrypted, so Apple itself cannot read them.
- **`photos.db`** stays local on every laptop that builds one. It is never synced to iCloud and never uploaded by this tool.
- **The password** is stored as a PBKDF2-HMAC-SHA256 hash with a per-archive random salt — never as plaintext.
- **Tailscale traffic** is end-to-end encrypted via WireGuard between your devices. Your password is never sent in cleartext over any network.
- **The viewer** makes no outbound connections. No telemetry, no analytics, no third-party services contacted.

- **What you commit to GitHub**: only the program source (text files). The `.gitignore` blocks `photos.db`, `ChatStorage.sqlite`, `Message/`, `Media/`, and the thumbnail cache by default, so even an accidental `git add .` won't leak your data.

Generated from `docs/QUICKSTART.md` by `docs/build_pdf.py`. Edit the Markdown and re-run the script to update the PDF.